

Realization document

Internship at Aigency

Ondrej Dobiš
Applied Computer Science

Table of Contents

TABLE OF CONTENTS	2
1. INTRODUCTION	5
1.1 Scope, Boundaries, and Reader Orientation	5
1.2 Working Methodology	7
1.3 Evolution of the Original Project Plan	8
2. ANALYSIS	9
2.1 Business Context	9
2.2 Existing System and Architecture	9
2.3 Business Case for New Features	10
2.3.1 Typical User and System Workflow	11
2.3.2 Stakeholders and Design Responses	11
2.4 Technologies Used	13
3. IMPLEMENTATION	16
3.1 Projects and Spaces Feature	16
3.1.1 Overview	16
3.1.2 Database Schema	16
3.1.3 Source File Storage	17
3.1.4 Context Injection	17
3.1.5 User Interface Design	18
3.2 AI-Driven Select Choices	18
3.2.1 Overview	18
3.2.2 Implementation with Vercel AI SDK	18
3.2.3 The Stale ItemId Bug	19
3.3 Interactive Slides System	19
3.3.1 Overview and Architecture	19
3.3.2 Slide Schema and Data Model	20
3.3.3 Generation Pipeline	20
3.3.4 Voiceover Generation	21
3.3.5 Template System and Responsiveness	22
3.3.6 Image Generation and Stock Photos	22
3.4 Quiz System	23
3.4.1 Overview	23
3.4.2 Database Schema	23
3.4.3 Generation	23
3.4.4 User Experience Details	24

3.5 Interactive Slides Editor	24
3.5.1 Overview	24
3.5.2 Inline Text and Table Editing	24
3.5.3 AI Assistant in the Editor	24
3.5.4 Chart Editing	25
3.5.5 Image Gallery and Cropping	25
3.6 Export Pipeline Enhancements	25
3.6.1 PDF Export	25
3.6.2 Video Export	26
3.6.3 Security and Billing Fixes	26
3.7 Internationalisation	27
3.7.1 Architecture	27
3.7.2 Scope of Translation	27
3.7.3 Register Normalization	27
3.7.4 Email Template Localization	28
3.8 Share and Analytics System	29
3.8.1 Overview	29
3.8.2 Access and Sharing Controls	29
3.8.3 Analytics Implementation	29
3.9 Extended Editor and Chart System	29
3.9.1 Chart and Table Capabilities	29
3.9.2 Inline Editing and Undo Support	30
3.9.3 Layout and Cache Reliability	30
3.10 Custom AI Presentations	30
3.10.1 Purpose and Architecture	30
3.10.2 Generation Contract and Progress Feedback	30
3.10.3 Hybrid Editor	30
3.11 Publishing, Export, and Production Reliability	31
3.11.1 Publishing Safety	31
3.11.2 Export Improvements	31
3.11.3 Production Monitoring and Fixes	31
3.12 Testing and Code Quality	31
3.12.1 Unit Tests	31
3.12.2 Manual Testing	32
3.12.3 AI-Assisted Code Review	32
4. API ENDPOINT OVERVIEW	33
5. FINAL STATUS AND REMAINING FOLLOW-UP	35
5.1 What Was Delivered	35

5.2 How the Project Can Be Used After the Internship	36
5.3 Remaining Follow-Up Areas	37
6. FUTURE STEPS	39
6.1 Real-Time Collaborative Editing	39
6.2 Advanced Engagement Intelligence	39
6.3 White-Label and Embedding	39
6.4 Multi-Modal Input	40
7. EVALUATION	41
7.1 What Went Well	41
7.2 What Was Difficult	41
7.3 Lessons Learned	42
7.4 Design Tradeoffs and Rejected Alternatives	42
7.5 Quality and Risk Management	43
7.6 Competencies Demonstrated	44
8. CONCLUSION	45
REFERENCE LIST	46
GENERATIVE AI ACKNOWLEDGMENT	47

1. Introduction

This document presents the practical realisation of my internship assignment at Agency. It follows the project plan submitted at the beginning of the internship and provides an academic account of the work I completed, the technologies I analysed and applied, and the features I developed for the host company's platform.

Agency is a start-up that currently develops an AI-powered presentation generation platform. The platform allows users to create, customise, and export presentations through a conversational AI interface, with output formats including PDF, video, and browser-based interactive slide decks. The product is intended for knowledge workers, trainers, and marketing teams that need to produce professional presentations efficiently without relying on an external design agency. At the start of the internship, the platform had a functional core, but several areas still limited its use at scale. These included workspace organisation, a live interactive output format, analytics and sharing capabilities, localisation, and richer post-generation editing.

Over the sixteen weeks of the internship, I substantially extended the scope of the platform. I implemented a projects and spaces management system so users could group conversations, source files, and presentations into named workspaces. I then built an interactive slides module that produces animated browser-based presentations rather than only static files. On top of this module, I added quiz functionality, share and analytics features, a rich in-browser editor, PDF and video export flows, and support for custom AI-generated presentation variants. I also internationalised the dashboard and related user-facing flows into Slovak and English.

Chapter 2 presents the analysis phase, including the existing architecture, the business context, the user and system workflow, stakeholder needs, and the technologies selected. Chapter 3 describes the main implementation work organised by feature domain, including architectural decisions, code-level details, and the most relevant challenges encountered. Chapter 4 gives an overview of the API endpoint surface. Chapter 5 summarises the final status of the planned work, the delivered value, and the remaining follow-up areas. Chapter 6 outlines future development opportunities beyond the internship period. Chapter 7 evaluates the internship work, including what went well, what was difficult, design tradeoffs, risk management, competencies demonstrated, and the main lessons learned. Chapter 8 provides the conclusion and final reflection.

1.1 Scope, Boundaries, and Reader Orientation

The scope of this realization document is the product and engineering work I can explain and defend from the internship period. It covers the business context,

architectural decisions, implementation reasoning, technical behaviour, testing approach, production issues, and reflection on the main tradeoffs. It does not reproduce credentials, customer data, private business conversations, or confidential repository details beyond what is necessary to explain the work academically. Several implementation details are therefore described through architecture, schemas, API flow, storage behaviour, background jobs, and user-facing effects rather than by copying large sections of internal source code. This is appropriate because LectureGuru is an active product sold as a service, including to teams and organisations with enterprise requirements. The purpose of the report is to show what was built, why those decisions were made, and how the system behaves technically.

Because the report contains several product-specific and technical terms, the most important ones are defined below before they are used repeatedly in later chapters.

Term	Meaning in this report
Interactive presentation	A browser-based presentation that stores slide content as structured data and can include animations, voiceovers, quizzes, sharing, analytics, and export paths.
Project context	Custom instructions, branding settings, uploaded source material, and related project data that are made available to the generation workflow.
Editor overrides	Saved post-generation changes such as moved elements, edited text, chart changes, image updates, or cropped media, kept separately from the original generated slide content.
Background job	A long-running operation, such as PDF export, video rendering, audio generation, or image generation, that runs outside the initial browser request and reports progress through status handling.
Private R2 asset access	A storage pattern where generated or uploaded assets remain in Cloudflare R2 and are accessed through signed download URLs or same-origin proxy routes, depending on the media type and playback requirements.
Credit reservation	A billing safeguard where credits are reserved before an expensive action and then consumed on success or released if the job fails or is cancelled.
Change Detective	A feature that detects selected web resources for updates and helps keep generated learning content aligned with the latest source information.

Magic Video

An AI demo-creation feature that records a browser workflow from a URL and task prompt, turns it into a polished narrated tutorial video, and saves it as a reusable asset for training, onboarding, support, and slide-based learning content.

1.2 Working Methodology

My working methodology was iterative. I first studied the existing application, database schema, generation flow, export logic, storage handling, and user interface. I often discussed priorities with the internship mentor and product direction, identified the business value of the requested feature, defined the relevant data or API contract, and implemented the work in smaller increments that could be reviewed, tested, adjusted and later deployed as a single unit of functionality.

In practice, each major feature moved through a similar cycle: analyse the existing behaviour, define the contract, implement the smallest usable version, test the happy path and edge cases, incorporate review feedback, and then harden the feature for production use. This method was especially important for projects, interactive slides, editor behaviour, exports, credits, analytics, and R2 asset handling, because each visible product improvement depended on several backend and infrastructure decisions.

The approach also helped with frequent product changes. Instead of treating changing priorities as a reason to restart, I tried to keep each layer understandable and extendable: project context remained separate from generated slides, editor overrides remained separate from original content, background jobs handled long-running work, and credit reservations protected expensive operations from billing mistakes.

1.3 Evolution of the Original Project Plan

The original project plan was prepared at the beginning of the internship, when LectureGuru was still in an early product phase within a start-up environment. At that point, the main direction was clear: improve the organisation of sources and presentations, support project-level context, explore automatic updating of learning material, introduce more interactive React-based presentation formats, and add features such as public or private access, voiceover, language support, quizzes, and analytics. However, the exact shape of these ideas could not be fully fixed before implementation started, because the product itself was still developing and new technical and commercial insights appeared during the internship.

For that reason, the project plan functioned as a guiding framework rather than a rigid checklist. The projects and spaces feature was implemented as planned and became the foundation for grouping sources, chats, personalization settings, and generated outputs. The idea of monitoring changes in project sources also remained relevant, because the product still needed a way to keep learning content aligned with updated material. In practice, this idea became connected to the broader project-context and source-management workflow instead of being treated as a separate isolated feature. This was appropriate because source updates, generation context, presentation regeneration, and user-facing outputs all depend on the same project-level data model.

The largest evolution concerned the planned React-based presentation format. During implementation, this grew into a broader interactive slides ecosystem. The work no longer involved only rendering slides with React components; it also required structured slide data, browser playback, templates, animations, voiceover, quizzes, sharing, analytics, editor support, and export paths for PDF and video. Several items from the original timeline, such as public or private access, voiceover, language toggle, quiz functionality, and analytics, were therefore delivered as connected parts of this larger product flow.

Additional work also became necessary as the feature set moved closer to production use. Editor reliability, export behaviour, private Cloudflare R2 asset access, credit reservations, billing safeguards, cache invalidation, and route security were not all described in detail in the initial project plan, but they became important once the new functionality had to work reliably for real users. These tasks were not separate from the internship goal; they were part of turning the planned features into usable product functionality.

Overall, the final scope followed the main direction of the original plan while adapting to the realities of an early-stage start-up product. The changes were handled through iterative implementation, discussion with the mentor, testing, review feedback, and prioritisation of work that created the most product value. This made it possible to keep the original objectives in view while allowing the details to evolve as the system and product understanding matured.

2. Analysis

2.1 Business Context

LectureGuru was started on the premise that producing a professional presentation or a training material should not require extensive manual work. The product allows a user to upload a document or paste a URL, describe the required presentation, and receive a designed deck within a few minutes. This places the company in the growing market of AI productivity tools, alongside products such as Gamma, Beautiful.ai, and Tome.

The product's main differentiators are the interactive slides module, change detective and magic demo features. Instead of exporting a file that must be opened in PowerPoint or Google Slides, the interactive output runs directly in the browser with animations, voiceovers, quiz slides, sharing, and analytics. This gives sales teams and trainers a hosted link that can be distributed and monitored more easily than a static attachment.

From a commercial perspective, the platform operates on a credit system. Users purchase monthly subscription plans, which contain a certain amount of tokens which are deducted for actions such as slide generation, image generation, voiceover generation, and video export. A significant part of my work involved billing and access-control infrastructure, because these features had to ensure that credits were deducted correctly and that free-tier users could not access paid-tier functionality through edge cases in authentication or route handling. An example paid feature is change detective, which is available in limited settings in the Individual subscription package.

2.2 Existing System and Architecture

During the first two weeks of the internship, I studied the existing codebase, tested the available features, and analysed the application data flow. At that stage, the platform was operational and consisted of the following layers:

- Front-end: Next.js 14 App Router with React Server Components, Tailwind CSS, and the shadcn/ui component library.
- Back-end: Next.js API routes and server actions handling generation requests, export requests, and data management.
- Database: PostgreSQL on Supabase, accessed through Drizzle ORM. The schema contained tables for users, conversations, messages, and generated presentations.
- Storage: Cloudflare R2 for generated PPTX and PDF files.

- AI layer: OpenAI API accessed through the Vercel AI SDK. The core generation loop used a two-step process: first generating a presentation outline, then generating slide content slide-by-slide.
- Authentication: Supabase Auth with row-level security policies on the database tables.

The most important structural finding from the analysis phase was that all content existed in a flat global scope. Conversations were not grouped, source documents were not saved for later use as context, and users had no way to apply custom branding or instructions to a group of presentations. I therefore worked on the projects feature as the first major implementation area, because later work on interactive slides, quizzes, exports, and analytics would benefit from project-level settings.

I also identified an issue in the presentation preview component: manual edits made in the preview interface were not included during export. I corrected this early in the internship so the existing export flow remained reliable while the new interactive slides work began.

2.3 Business Case for New Features

Before starting each major workstream, I discussed the features and prioritization with the internship mentor to get aligned on what to do, technical feasibility, and expected user benefit. The business rationale behind the main features was as follows:

Projects and Spaces: Power users can quickly accumulate many conversations and presentations without a reliable way to organise them. The projects feature addresses this by grouping related content together. Custom instructions and branding at the project level also reduce repeated prompting, which improves generation quality and saves time.

Select Choices: Presentation quality declined when users provided vague prompts. The select-choices flow improves this by asking the user to clarify presentation length, topic, target audience and text density before generation begins. The intended result was fewer revision cycles and a more predictable first output.

Interactive Slides: Static exports such as PPTX and PDF require the recipient to open a file locally. A browser-based format with voiceovers, quiz slides, sharing, and analytics creates a different product experience because it can be accessed through a link on any device and improved based on viewer behaviour. This became the largest feature area of the internship.

Quiz System: Training and onboarding teams were an important target segment. Quizzes embedded directly in a presentation, with results shown inside the deck,

made the product more useful for learning-oriented use cases than tools that only support static content.

Internationalisation: The company operates primarily in Slovakia and needed the dashboard, generated content, and email flows to work in Slovak as well as English. Without this, the product remained effectively Slovak-only, which limited adoption in new markets.

2.3.1 Typical User and System Workflow

The completed platform workflow begins when a user creates a project, defines custom instructions, and uploads source material such as documents, presentation files, or web content. The system stores this material as project context and makes it available to the generation pipeline whenever a presentation is created inside that workspace. This design means that the user does not have to repeat company-specific instructions, brand preferences, or source context in every conversation. A typical end-to-end flow is therefore: create a project, upload sources, start a generation request, answer AI-generated clarifying questions, produce an interactive presentation, refine it in the browser editor, add quizzes or voiceovers where appropriate, share or export the final result, and then inspect viewer activity through analytics. From the user's perspective this is a single product flow, but technically it involves several coordinated layers: project context injection, structured OpenAI generation through the Vercel AI SDK, database persistence of presentation and slide records, editor overrides stored separately from generated content, background export jobs, Cloudflare R2 storage for generated assets, signed URLs for secure file access, and analytics events recorded during playback.

This workflow became important because Aigency is selling the platform mostly to enterprise clients as a service. These clients need more than a one-off slide generator: they need repeatable branding, controlled sharing, reliable exports, and measurable engagement. The workflow implemented during the internship supports that direction by connecting generation, editing, delivery, and reporting into one coherent product cycle.

2.3.2 Stakeholders and Design Responses

The feature decisions during the internship were influenced by several stakeholder groups. The public LectureGuru website positions the product for teams and organisations that need to create learning materials quickly, with examples including HR, internal communication, IT/helpdesk, education, legal and compliance, customer support, and professional associations. The pricing page also describes an enterprise plan with multiple users, multiple workspaces, team roles, shared templates, API and webhook access, and priority support. These expectations shaped the technical priorities of the internship work.

Stakeholder	Expectation	Design response
Enterprise customers	Multiple users, workspaces, roles, shared templates, API/webhook readiness, reliable exports, and controlled delivery.	Projects, share controls, analytics, export flows, private asset access, and production reliability fixes support enterprise service requirements.
Presenters and trainers	Professional materials that can teach, test knowledge, narrate, and be distributed without requiring local presentation software.	Interactive slides, voiceovers, quizzes, hosted playback, and MP4 /PDF export paths support training and presentation workflows.
HR and internal communication teams	Fast creation of onboarding, compliance, benefits, and policy-training material from ideas or documents.	The AI assistant, source uploads, project context, and generated slide/video outputs reduce repeated manual preparation work.
IT/helpdesk and support teams	Tutorials, feature explanations, and support material that can be produced quickly and updated as source material changes.	Structured generation, Change detective/source workflows, export formats, and shareable outputs support repeatable operational communication.
Marketing, sales, and product teams	Editable generated material that can be polished, branded, shared, and measured.	The editor, project-level branding/context, sharing, analytics, custom presentations, and export pipeline bridge generation and professional delivery.
Founder / product owner	Fast delivery of commercially useful features without making the platform fragile.	Incremental implementation, code review, validation, cache fixes, billing fixes, and background-job boundaries kept feature growth tied to production safety.
Developers and maintainers	A codebase that stays understandable as AI, editor, export, analytics, and billing behaviour grow together.	Schema validation, route organization, separated editor overrides, Trigger.dev tasks, credit reservation services, and tests support maintainability.

2.4 Technologies Used

The following table summarises the main technologies used during the internship, the purpose of each technology, and the reasoning behind its selection.

Technology	Purpose	Rationale
Next.js 14 (App Router)	Full-stack framework	Already part of the stack. The App Router enables React Server Components, which reduce client bundle size and simplify data-fetching patterns. API routes and server actions live in the same repository as the UI, keeping the stack unified.
Drizzle ORM + Supabase PostgreSQL	Database layer	Drizzle provides type-safe schema definitions and a migration workflow that integrates well with TypeScript. Supabase adds hosted PostgreSQL with branch databases, which proved valuable for testing schema changes in isolation before merging to main.
Cloudflare R2	Object storage	S3-compatible storage for uploaded source files, generated images, audio files, and export artefacts. Significantly cheaper than S3 at the usage volumes involved, with no egress fees and built-in signed-URL support.
Vercel AI SDK	AI streaming and tooling	Abstracts streaming, tool-calling, and multi-step agent workflows over OpenAI and other providers. The useChat and streamText APIs made it straightforward to build the select-choices flow and the in-editor AI assistant.
OpenAI API (GPT-4o)	Text generation	Chosen by the company as the primary language model provider for its balance of generation quality and latency.
ElevenLabs API	Text-to-speech voiceovers	High-quality neural voices across many languages. Evaluated against Amazon Polly and Google TTS; ElevenLabs produced noticeably more natural-sounding speech, especially in Slovak.
Remotion	Video rendering	React-based programmatic video generation. Renders React components frame-by-frame into MP4, enabling chart animations and

Unsplash API	Stock imagery	slide transitions in video exports without requiring a headless browser at runtime. Provides license-free images fetched dynamically based on slide content. Integrated with mutual-exclusivity logic so image and chart slots do not conflict on the same slide.
Flux 2.0	AI image generation	Generates contextually relevant images when no suitable stock photo is available. Runs asynchronously and stores results in R2.
next-intl	Internationalisation	Provides a simple, Next.js-native i18n solution with cookie-driven locale management, compatible with both server and client components.
Trigger.dev	Background jobs	Handles long-running export tasks (video rendering, PDF generation) outside the main request cycle, with progress reporting to the front-end via database polling.

Most of these technologies were already part of the stack at the start of the internship. ElevenLabs and Remotion were the main additions. I evaluated ElevenLabs against Amazon Polly and Google TTS, and it produced more natural speech, especially in Slovak. Remotion was chosen over headless-browser approaches such as Puppeteer because it renders React components frame by frame into video, preserving slide animations as they appear in the browser. Trigger.dev was used for background export jobs because it supports long-running tasks outside the main Next.js request cycle, including retry handling and status polling.

The choice of ElevenLabs was primarily driven by voice quality and multilingual support. The interactive slides module needed narration that would sound acceptable in English, Slovak and other supported languages because synthetic speech that feels unnatural quickly reduces the perceived quality of a presentation. Compared with more traditional text-to-speech providers, ElevenLabs produced more natural pacing and intonation in the tested use cases, which made it the better fit for a product sold as a polished presentation service.

Remotion was selected because the interactive slides are implemented as React components with animation logic. Rendering those components frame by frame into video preserves the visual behaviour of the browser presentation more reliably than a headless-browser recording approach. This was especially relevant for enterprise delivery, where exported videos must look consistent and professional without depending on a viewer's browser or network conditions.

Trigger.dev and Cloudflare R2 were important for reliability and security rather than only for feature completeness. Long-running PDF, video, and voiceover tasks should not block the browser request that starts them, so they are queued as background jobs and tracked through status endpoints. Generated files are stored privately in R2 and exposed through short-lived signed URLs, which protects user assets while still allowing downloads and playback when the authenticated workflow requires them. The Vercel AI SDK and OpenAI integration were used because the product needed structured generation rather than only free-form text completion. Tool calling, schema-guided output, and streamed responses made it possible to build flows such as clarifying choices, slide-by-slide generation, editor assistance, and repair of malformed AI output. This was an important engineering constraint: the AI layer had to produce data that could be validated, stored, edited, exported, and safely recovered when the model returned incomplete or unexpected fields.

3. Implementation

This chapter describes the main features I developed during the internship. Each section explains what was built, the architectural approach used, relevant code-level details, and the specific challenges that influenced the final implementation.

3.1 Projects and Spaces Feature

3.1.1 Overview

The first major feature I implemented was a projects module that allows users to organise conversations, source files, and generated presentations into named workspaces. Each project stores a name, custom AI instructions, a preferred font and colour scheme, logo, and a collection of source documents. When a user generates a presentation attached to a project, the custom instructions and uploaded sources are automatically included as context in the generation prompt.

3.1.2 Database Schema

I added a projects table and a project_sources table to the database schema. The projects table stores the project name, owner user ID, custom instructions, preferred font and colour scheme, and timestamps. The project_sources table stores metadata for attached files, including the R2 storage key, original filename, MIME type, and a foreign key to the project. I also added a foreign key from the chats table to projects so each conversation can be associated with a project.

The Drizzle schema definition for the projects table looks like this:

```
export const projects = pgTable('projects', {
  id: uuid('id').defaultRandom().primaryKey(),
  userId: uuid('user_id').notNull().references(() => users.id, { onDelete: 'cascade' }),
  name: varchar('name', { length: 255 }).notNull(),
  instructions: text('instructions'),
  font: varchar('font', { length: 100 }),
  primaryColor: varchar('primary_color', { length: 7 }),
  secondaryColor: varchar('secondary_color', { length: 7 }),
  createdAt: timestamp('created_at').defaultNow().notNull(),
  updatedAt: timestamp('updated_at').defaultNow().notNull(),
});
```

I used the project database command “pnpm --filter database push”, which applied the Drizzle schema changes directly to the target database. After the push completed, I verified the resulting schema and then merged the branch into main.

3.1.3 Source File Storage

Source files uploaded to a project are stored in Cloudflare R2 under a per-user prefix. The storage key is structured as `users/{userId}/projects/{projectId}/sources/{uuid}-{originalFilename}`. This keeps files organised by user and project and avoids key collisions when two users upload files with the same name.

Files in R2 are private by default. The initial implementation linked directly to the raw R2 object URL, which is not accessible without authentication. This created a bug where users could not download their own uploaded files. I fixed this by adding a download API endpoint that generates a signed URL with a short expiry:

```
// GET /api/projects/[id]/sources/[sourceId]/download
const signedUrl = await r2.sign(
  new Request(objectUrl, { method: 'GET' }),
  { accessKeyId, secretAccessKey, expiresIn: 300 }
);
return Response.redirect(signedUrl.url);
```

A five-minute expiry was chosen as a reasonable balance between security and usability. The URL is regenerated on each download request, so there is no need to manage expiry on the client side.

3.1.4 Context Injection

When a user sends a message in a project-linked conversation, the generation API fetches the project’s custom instructions and source documents. Source documents are included in the prompt as extracted text content. For PPTX and DOCX files, text is extracted server-side using the mammoth and pptx-parser libraries. For PDF files, pdf-parse is used. For web URLs, a lightweight fetch-and-strip-HTML step runs at upload time and the plain text is stored alongside the source record.

The custom instructions and source text are injected as a system message prefix before the conversation history. This means the model always has the project context available regardless of how many messages are in the conversation.

3.1.5 User Interface Design

I redesigned the projects interface with a focus on a cleaner visual hierarchy, simpler navigation, and a more natural flow between related pieces of information. The redesign was guided by feedback from the internship mentor and by findings from manual testing.

A specific UX improvement was the addition of presentation grouping in the project detail view. When multiple interactive presentations originate from the same chat or share a name, they are collapsed into a single card showing a version count. Expanding the card reveals each version as a row. This reduced visual clutter in projects that had gone through several generation iterations. The version grouping is computed on the API side before the data is returned to the client, so the front-end only needs to render the grouped structure rather than deduplicate it.

I also improved the personalisation settings tab by replacing the manual Save button with an autosave hook. Previously, users could navigate away without saving. The new hook debounces changes by 1.5 seconds and writes them to the API on the trailing edge. A small status indicator shows “Saving...” during the debounce period and “Saved” after the API call completes.

3.2 AI-Driven Select Choices

3.2.1 Overview

When a user starts a presentation, the AI now pauses after the initial message and presents a set of clarifying choices. These choices cover the parameters with the strongest influence on generation quality, including presentation length, target audience, explanation depth, and topic scope. The user can select one of the proposed options or type a custom answer. The selected values are then included as structured context in the next generation step.

3.2.2 Implementation with Vercel AI SDK

I implemented this feature with the Vercel AI SDK’s tool-calling mechanism. The first generation step calls the `presentChoices` tool, which streams choice objects to the client. The client renders them as buttons in the chat interface. When the user selects a choice, it is sent as a new user message and the generation pipeline continues with the added context.

The relevant part of the server-side stream setup looks like this:

```
const result = streamText({  
  model: openai('gpt-4o'),
```

```

system: SYSTEM_PROMPT,
messages: conversationHistory,
tools: {
  presentChoices: tool({
    description: 'Present clarifying choices to the user before generating.',
    parameters: z.object({
      choices: z.array(z.object({
        label: z.string(),
        value: z.string(),
        description: z.string().optional(),
      })),
    })),
  },
},
});

```

This feature required careful prompt engineering. The first version sometimes skipped the choices step if the user's initial message was already detailed. I improved the behaviour by adding explicit instructions to present choices unless the user had already answered all required parameters in the message.

3.2.3 The Stale ItemId Bug

During testing, I found an intermittent 404 error after a user had completed the select-choices flow in the same conversation. The root cause was that the OpenAI Responses API stores item IDs for tool calls in the conversation history. When the chat was rebuilt from the database for a later request, some assistant messages still contained those item IDs. If the original OpenAI session had expired, the API returned a 404 when attempting to reference them.

I fixed this by stripping item IDs from persisted assistant messages before passing them back to the model. A utility function scans the conversation history and removes itemId fields from tool-call messages before the array is sent to the API.

3.3 Interactive Slides System

3.3.1 Overview and Architecture

The interactive slides module was the largest single feature I developed during the internship. Instead of generating only a static file, the system produces a fully animated browser-based presentation that runs as a React application. Each presentation is stored as structured JSON in the database: a top-level record contains metadata, and an ordered set of slide records contains type identifiers, content fields, and optional editor overrides.

The system supports four visual templates, each implemented as a self-contained React component tree. Each template has its own colour palette, typography, and animation style. Within a template, slide types map to specific layouts, including title slides, content slides, chart slides, image slides, quote slides, list slides, glossary slides, and long-form text slides.

3.3.2 Slide Schema and Data Model

I represented each interactive presentation with two database tables: `interactive_presentations` and `interactive_slides`. The presentations table stores the title, template name, language, project ID, user ID, and generation status. The slides table stores the slide type, content as JSONB, the slide index, and an overrides JSONB column that records editor changes.

The content schema varies by slide type but follows a consistent envelope:

```
type SlideContent = {
  heading?: string;
  subheading?: string;
  body?: string;
  bullets?: string[];
  chart?: { type: 'bar' | 'line' | 'area' | 'pie'; data: ChartData };
  image?: { src: string; alt: string; source: 'unsplash' | 'flux' | 'upload' };
  quote?: { text: string; attribution: string };
  tableRows?: string[][];
  terms?: { term: string; definition: string }[];
};
```

This schema allows a single rendering function to handle all slide types without needing to know the template in advance. The template component reads the type field and delegates to the appropriate layout sub-component.

3.3.3 Generation Pipeline

Generating an interactive presentation uses two OpenAI calls. The first call creates a presentation plan containing the overall structure, slide count, slide types, and a short description of each slide. The second call iterates over that plan and generates the full slide content. Both calls use structured output through the Vercel AI SDK's `generateObject` function, which ensures that the model returns JSON matching the expected schema.

The two-phase approach was necessary because a single call for presentations of 30+ slides consistently hit context limits or produced inconsistent output quality in the

later slides. Splitting the work into planning and execution gave the model a clearer, more focused task in each call.

The generation endpoint is a POST request to `/api/interactive/generate`. It accepts the following parameters:

- `projectId`: the project to associate the presentation with.
- `prompt`: the user's description of what the presentation should cover.
- `template`: the template identifier (e.g. 'modern', 'corporate', 'minimal', 'bold').
- `slideCount`: the requested number of slides (1 to 150).
- `language`: the output language code (e.g. 'en', 'sk', 'de').
- `includeImages`: whether to generate AI images or fetch from Unsplash.
- `includeVoiceovers`: whether to generate ElevenLabs voiceovers.
- `includeQuiz`: whether to append a quiz section.
- `density`: 'concise' for bullet-style slides or 'detailed' for reading-first text-heavy slides.

3.3.4 Voiceover Generation

Voiceovers are generated using the ElevenLabs API. For each slide that has a voice script, an API call is made to the ElevenLabs `/v1/text-to-speech/{voiceId}/stream` endpoint with the script text. The returned audio stream is uploaded to Cloudflare R2, and the R2 key is stored on the slide record. The browser-based presentation player reads the R2 key and plays the audio when the slide is active.

During testing, I found a concurrency issue in voiceover generation. When voiceovers were generated for all slides at the same time, ElevenLabs returned 429 rate-limit errors for some requests. I resolved this by adding exponential backoff with jitter:

```
async function generateWithBackoff(script, voiceId, attempt = 0) {
  try {
    return await elevenLabs.generate({ text: script, voiceId });
  } catch (err) {
    if (err.status === 429 && attempt < 5) {
      const delay = Math.pow(2, attempt) * 500 + Math.random() * 300;
      await sleep(delay);
      return generateWithBackoff(script, voiceId, attempt + 1);
    }
    throw err;
  }
}
```

3.3.5 Template System and Responsiveness

The four visual templates — Modern, Corporate, Minimal, and Bold — are each implemented as a self-contained React component tree. A template consists of a root wrapper that sets the colour palette, typography, and background, and a set of named slide-type components that the root delegates to based on the slide's type field. This separation means a new template can be added by creating a new root wrapper and a set of slide-type components without touching the generation pipeline or the database schema.

I tested each template at three breakpoints: desktop (1280px), tablet (768px), and mobile (375px). At tablet and mobile sizes, multi-column layouts reflow into a single column, font sizes scale down proportionally, and touch-based navigation replaces keyboard navigation. Making these breakpoints consistent across all four templates required several iterations, because the initial implementation worked on desktop but produced inconsistent spacing and truncated text on smaller screens.

I added animated auto-zoom in Week 9. When a presentation is displayed inside a container smaller than the slide's native resolution, the slide is scaled down with a CSS transform. The zoom factor is calculated from the container dimensions and applied through a useResizeObserver hook. When the container changes size, for example after a phone rotation, the zoom factor updates smoothly with a short CSS transition.

3.3.6 Image Generation and Stock Photos

Each slide in the plan is tagged by the first generation phase as image-eligible or not. For image-eligible slides, the system decides between two sources. If the slide content is factual and a relevant stock photo is likely to exist, the Unsplash API is queried with a keyword derived from the slide's heading. If a good match is found, the image URL is embedded directly. If not, or if the content is conceptual, the Flux 2.0 API generates an image from a short prompt.

Charts and images share a slot in the slide layout. If the generation plan allocates a chart to a slide, no image is requested for it. This mutual exclusivity was originally enforced only in the prompt but was later also enforced in the validation layer to prevent the model from returning unsupported data with both fields populated simultaneously.

3.4 Quiz System

3.4.1 Overview

The quiz system adds three new slide types to the interactive slides framework: quiz-hero (an introductory splash screen for the quiz section), quiz-question (a multiple-choice question slide), and quiz-results (a summary slide showing score and feedback). Quizzes are appended to the end of an interactive presentation when the `includeQuiz` flag is set.

3.4.2 Database Schema

The quiz schema was added as part of a migration. Quiz questions are stored in a `quiz_questions` table with columns for the presentation ID, slide index, question text, and a JSONB array of answer options. Each option has a text field and a correct boolean. The results slide reads from this table at runtime to calculate the user's score.

```
export const quizQuestions = pgTable('quiz_questions', {
  id: uuid('id').defaultRandom().primaryKey(),
  presentationId: uuid('presentation_id').notNull()
    .references(() => interactivePresentations.id, { onDelete: 'cascade' }),
  slideIndex: integer('slide_index').notNull(),
  questionText: text('question_text').notNull(),
  options: jsonb('options').notNull().$type<QuizOption[]>(),
});
```

3.4.3 Generation

I separated quiz generation from the main slide generation pipeline. After the main slides are generated, a dedicated quiz-assembly prompt uses the presentation topic and slide content to generate three to eight questions with four answer options each. The prompt instructs the model to base questions on the actual slide content rather than introducing unsupported facts.

One reliability issue was that the model occasionally generated questions referencing content that was cut off or abbreviated in the slides. This was mitigated by including the full slide content (not just headings) in the quiz-assembly prompt, which increased token usage slightly but substantially improved question quality.

3.4.4 User Experience Details

I made several user-experience decisions during quiz development. The results screen remains locked until all questions have been answered, and attempts to open it early redirect the user to the first unanswered question. This prevents users from skipping directly to the score. A subtle “next” arrow appears on answered quiz slides after 15 seconds to guide the user forward. I also added a project-level toggle so quizzes can be disabled for projects where they are not appropriate.

3.5 Interactive Slides Editor

3.5.1 Overview

The editor allows users to modify a generated interactive presentation directly in the browser without regenerating it. It is accessed from the presentation detail page and shows a live preview of the active slide alongside editing panels. Edits are persisted to the database and reflected in the live presentation URL.

3.5.2 Inline Text and Table Editing

Inline text editing is implemented with a contenteditable HTML overlay positioned on top of the rendered slide element. When a user double-clicks a text element, the overlay becomes active and the user can type directly on the slide surface. On blur, the new text content is extracted and saved. Tables use the same overlay with cell-level contenteditable regions.

A challenge with this approach is that the contenteditable element must be pixel-accurately positioned over the rendered slide element. Because the slide is scaled to fit the viewport using a CSS scale transform, the overlay coordinates must account for the scale factor. The editor reads the scale factor from a React context and applies the inverse transform to the overlay’s position calculations.

3.5.3 AI Assistant in the Editor

The editor includes an AI chat panel. Users can describe a change in natural language, such as changing a heading or adding a chart, and the assistant applies the requested change. I added dedicated tools for the editor assistant, including `get-slide`, which fetches the current state of a slide, and `read-source`, which retrieves text from an uploaded source document. These tools give the model access to context beyond the active slide.

During development, the editor assistant sometimes returned malformed tool calls, for example with missing required fields or incorrect field names. I added a validation and repair layer in the API route that checks tool output against the slide schema

before applying it. If required fields are missing, the system infers reasonable defaults from the existing slide content instead of immediately failing.

3.5.4 Chart Editing

Charts in the editor are editable through a properties panel that appears when a chart element is selected. The panel shows the chart type, a table of editable data series, and colour controls. When the user commits a change, the slide content is updated in the database and the chart component is remounted. Remounting was necessary because the Remotion-based animation runs on mount; updating props alone skipped the entry animation.

I also fixed a colour-picker bug that reset the selected colour on every render. The problem was that the colour state was initialised from slide content on mount but was not kept in sync with database updates after the first save. I resolved this by moving colour state into the parent editor context and passing it as a controlled prop.

3.5.5 Image Gallery and Cropping

The image gallery in the editor shows all images available for the active presentation: AI-generated images, Unsplash images, and any images the user has manually uploaded. Images are grouped by presentation. A cropping tool is integrated directly into the gallery: selecting an image opens a crop dialog with drag handles, and the cropped output is uploaded to R2 as a new object. The original image is preserved.

During testing, I found that Flux-generated images stored in R2 did not load on mobile devices or for unauthenticated viewers. The embedded image URL pointed to a private R2 path. I fixed this by passing image URLs through the same signed-URL generation flow used for source files and refreshing the signed URL whenever the presentation is fetched.

3.6 Export Pipeline Enhancements

3.6.1 PDF Export

PDF export for interactive slides works by rendering the presentation in a headless Chromium instance (via Playwright), printing each slide to a PDF page, and concatenating the pages using pdf-lib. The rendering runs as a Trigger.dev background job to avoid blocking the API response. The completed PDF is uploaded to R2 and the user is notified by email.

The most technically demanding part of the PDF export was handling slides that use CSS animations. Playwright can capture a slide while animated elements are still in their initial positions, which may place them outside the visible area. I fixed this by injecting a CSS rule before capture:

```
await page.addStyleTag({
  content: '*', '::before, ::after { animation-play-state: paused !important;
    animation-delay: -9999s !important; }'
});
```

This forces all animations to their final frame, producing clean screenshots of fully-rendered slide content.

3.6.2 Video Export

Video export uses Remotion, which renders each slide as a sequence of React component frames and assembles them into an MP4. The slide transitions are defined as Remotion sequences with spring animations. Charts have dedicated Remotion animation variants that play on entry.

Initial video export performance was insufficient: exporting a 20-slide presentation took more than three minutes. The main bottleneck was sequential generation of chart-animation screenshots and audio waveform assets. Switching to concurrent processing with Promise.all reduced the export time to under one minute for the same presentation:

```
const [screenshots, audioAssets] = await Promise.all([
  captureAllSlideScreenshots(slides),
  downloadAllAudioFiles(slides),
]);
```

3.6.3 Security and Billing Fixes

During the export work, I identified and fixed several security and billing issues. One issue was a credit-reservation leak: credits were reserved when an export job started but were not released if the job failed. I added a reservation timeout so credits are released after a maximum job duration. I also found an authorisation gap where an export endpoint checked only that a presentation existed, not that the requesting user owned it. I added row-level ownership checks to the export endpoints. Finally, I resolved a tier-resolution bug where paid users could be identified as free-tier users because Stripe product identifiers were being compared against price identifiers.

3.7 Internationalisation

3.7.1 Architecture

The internationalisation implementation uses next-intl with a cookie-driven locale lifecycle. The selected locale is stored in a locale cookie and read by Next.js middleware at request level. I created a shared packages/i18n package in the monorepo to hold translation catalogues and locale helpers, allowing both the main app and email templates to import translations from a single source.

The middleware setup routes requests through Next.js's middleware chain before they reach the application:

```
// middleware.ts
import { createNextIntlPlugin } from 'next-intl/server';
const withNextIntl = createNextIntlPlugin();
export default withNextIntl(middleware);

// Inside the middleware function:
const locale = request.cookies.get('locale')?.value ?? 'en';
requestHeaders.set('x-locale', locale);
```

3.7.2 Scope of Translation

The translation work covered settings pages, authentication flows, the onboarding wizard, the files page, the home page, interactive slides UI strings, and all user-facing email templates. I also localised AI tool responses by passing the locale through the tool context and replacing hard-coded Slovak strings with locale-aware helper functions.

Invitation emails presented a specific challenge: the recipient's locale is unknown at send time because the invitation may be sent to someone who has not yet set up an account. The solution was to render both languages side by side in the same email, with a clear visual separator between the two language blocks.

3.7.3 Register Normalization

After the initial rollout, I normalised the Slovak translations from informal to formal register. Slovak distinguishes between informal and formal address, similar to the tu/vous distinction in French or du/Sie in German. The first translations used an informal register, which was not suitable for a business product. I therefore reviewed the translation catalogues and updated forms of address, verb endings, and pronoun references.

3.7.4 Email Template Localization

Email templates presented a separate localisation challenge from the in-app UI. Transactional emails (password reset, presentation delivery, export completion, invitation) are rendered on the server using React Email templates and sent via Resend. Localization was added by accepting a language parameter on each template component and switching between translation objects at render time, matching the same translation catalog structure used in the main application.

Invitation emails are a special case. When a user invites a colleague to join the platform, the recipient may not yet have an account and has not yet set a locale preference. Sending an English-only email to a recipient who speaks Slovak, or vice versa, produces a poor first impression. The solution was to render both language blocks side by side in the same email — Slovak on top, English below — with a thin horizontal rule between them. This is not ideal as a long-term design but is practical given that the platform targets a bilingual user base where most users are likely to understand at least one of the two languages.

Export completion emails include a signed download link to the generated file in R2. The link has a seven-day expiry, which was chosen based on typical download behaviour: most users download their export within the first hour, but a week gives enough buffer for users who check email infrequently. After the expiry, the file itself is not deleted, so users can regenerate a fresh link by visiting the presentation detail page.

3.8 Share and Analytics System

3.8.1 Overview

After the first version of interactive slides was stable, I implemented a share and analytics system so generated presentations could be distributed, controlled, and evaluated after publication. Authors can make a presentation public or private, generate a share link, invite viewers by email, and review engagement information from the same interface. This changed interactive slides from an isolated output format into a hosted delivery channel.

3.8.2 Access and Sharing Controls

The sharing workflow supports public links, private access, custom slugs, and email invitations. Private presentations require authentication and access checks before the viewer route opens. I also corrected post-login redirects so invited viewers return to the intended private presentation after signing in. These changes were important because a share link must behave predictably for both the author and the recipient.

3.8.3 Analytics Implementation

The analytics system records viewing and quiz-related events and presents them through tabs for share settings, views, and quiz results. The views tab shows recent unique-viewer activity and per-viewer progress. The quiz tab shows score indicators and per-viewer breakdowns. I excluded the creator's own views from the analytics so authors do not distort the data while testing their presentations.

Several reliability improvements were required after the first implementation. I added cache invalidation so new views and quiz attempts appear promptly, adjusted live refresh behaviour so the views and quiz tabs remain consistent, and moved share-link self-healing out of cached read paths. These changes reduced the risk of stale analytics or outdated links being shown to the author.

3.9 Extended Editor and Chart System

3.9.1 Chart and Table Capabilities

In the later part of the internship, I extended the editor from basic chart editing to a broader chart and table editing system. The chart catalogue was expanded to include additional chart types such as composed, scatter, radar, funnel, treemap, and radial charts. I also centralised the rules that define which slide types can support tables and how much content each template can safely display. This made the editor and renderer more consistent because they could rely on the same capacity rules.

3.9.2 Inline Editing and Undo Support

A major focus was making chart text editable directly on the canvas. Axis labels, chart titles, legends, funnel stages, treemap nodes, and segment labels can be edited in place. I also expanded the undo and redo history so changes to images, branding, colours, and presentation titles can be reverted in the same way as text edits. This made the editor feel more complete from the author's perspective.

3.9.3 Layout and Cache Reliability

I also addressed two long-standing reliability issues in the editor. First, I changed the slide preview so it can grow vertically when content exceeds the standard 1280 by 720 canvas, preventing authors from editing content that was hidden below the visible area. Second, I unified the cache-invalidation paths used after autosave, publishing, visibility changes, slug updates, image completion, and audio completion. This ensured that the live share link reflects author changes immediately instead of showing stale content.

3.10 Custom AI Presentations

3.10.1 Purpose and Architecture

The Custom AI presentation feature introduced a new generation path. Instead of choosing one of the fixed templates, the user describes the desired result and a background coding agent builds a bespoke interactive presentation. The generated deck is produced inside a sandboxed Preact and Vite project, validated, built, and published as an interactive presentation that appears in Files and Projects alongside regular interactive slides.

3.10.2 Generation Contract and Progress Feedback

I normalised inputs such as language, slide-count ranges, image preferences, and company names before they reach the background task. This prevents user choices from being lost or silently replaced by defaults. I also replaced raw agent logs with a milestone-based progress display, so users can understand the generation process without seeing implementation-level terminal output.

3.10.3 Hybrid Editor

I then implemented a hybrid editor for Custom AI presentations. Authors can make direct changes through a slide-by-slide editor backed by a versioned manifest, or they can open a dedicated edit chat and ask the agent to modify the deck. The live share link is only updated after a rebuild has successfully validated, which protects viewers from broken intermediate versions while the author is editing.

3.11 Publishing, Export, and Production Reliability

3.11.1 Publishing Safety

The later work on Custom AI presentations focused strongly on publishing safety. I added recovery for older generated source archives, corrected private-deck routing, and served viewer assets through a same-origin proxy with access checks. I also added browser-based smoke validation across generated decks so a presentation that crashes on a later slide is caught before publication.

3.11.2 Export Improvements

The export work was extended so Custom AI presentations could use stored export records and background rendering flows similar to regular interactive slides. Video export moved toward capturing the canonical published deck, making the output match what users see in the viewer. PDF export was added through the same stored export flow, and video sharpness was improved by capturing at a higher resolution before downsampling.

3.11.3 Production Monitoring and Fixes

The final weeks included several production-focused fixes. I added fallback polling when real-time audio task subscriptions drop, corrected schema requirements in the AI verification step, tightened public and private route behaviour, improved asset replacement from selected slide elements, and continued monitoring live analytics, exports, and Custom AI generation after deployment. These tasks were smaller individually, but together they made the new systems more dependable in production.

3.12 Testing and Code Quality

3.12.1 Unit Tests

I wrote unit tests throughout the internship using Vitest, which was already configured in the project. In practice, I usually wrote tests near the end of each feature workstream rather than alongside the first implementation. This was partly due to time constraints and partly because the API and data model often changed during early development. Writing tests once the shape had stabilised proved more efficient than repeatedly updating tests during exploration.

The main areas covered by unit tests were the CRUD API routes for projects and sources, the select-choices generation flow, stale itemId cleanup, credit deduction, tier-resolution logic, language-slot validation, autosave validation, and selected chart and editor behaviours. Database interactions were tested against a real Supabase branch database rather than mocked. This made the tests slower, but it also caught schema and integration issues that mock-based tests would likely have missed.

3.12.2 Manual Testing

Each feature workstream concluded with manual testing that covered the full user journey. For interactive slides, this included generating presentations with each template, switching languages, playing voiceovers, editing slide elements, exporting to PDF and video, and verifying that the exported files opened correctly. For the quiz system, I tested correct and incorrect answer flows, result locking, and navigation before all questions were answered. Later testing also covered share links, private invitations, analytics refreshes, Custom AI generation, and export validation.

Several important issues were found only through manual testing. These included the mobile image-loading issue, the density propagation bug that appeared only after a full generation cycle, a stale font default in template previews, private-route behaviour that depended on authentication state, and cache-invalidation issues that appeared only after saving and reopening shared presentations.

3.12.3 AI-Assisted Code Review

The company uses automated AI code review bots that comment on pull requests. Two bots were configured in the repository: one that scans for security issues (unauthorized access patterns, missing input validation, SQL injection risks) and one that reviews general code quality (dead code, missing error handling, overly complex functions, missing test coverage). Each pull request received comments from both bots before a human review.

Several of the most valuable fixes during the internship came from these automated reviews. The authorization bypass in the export endpoints was flagged by the security bot before the code reached human review. The credit reservation leak was surfaced as a missing try-finally pattern. The missing Zod validation on several API routes was flagged as a code quality issue. Applying these findings became a reliable final step before marking a pull request as ready for review.

4. API Endpoint Overview

This chapter summarises the main API routes introduced or modified during the internship. All routes require a valid Supabase session cookie unless otherwise noted. Routes that operate on user-owned resources include row-level ownership checks in addition to session validation.

Method	Route	Description
GET	/api/projects	List all projects for the authenticated user.
POST	/api/projects	Create a new project with name, instructions, theme, and font.
GET	/api/projects/[id]	Fetch a single project with its sources, chats, and presentations.
PATCH	/api/projects/[id]	Update project name, instructions, or personalization settings.
DELETE	/api/projects/[id]	Delete a project and cascade-delete all associated sources.
POST	/api/projects/[id]/sources	Upload a source file (PPTX, DOCX, PDF) to Cloudflare R2 and attach to project.
DELETE	/api/projects/[id]/sources/[sourceId]	Remove a source file from R2 and unlink from the project.
GET	/api/projects/[id]/sources/[sourceId]/download	Generate a signed Cloudflare R2 URL for authenticated file download.
POST	/api/chat	Core generation endpoint. Accepts message history, project ID, and settings; streams AI response via Server-Sent Events.
GET	/api/interactive/[id]	Fetch a generated interactive presentation with all slide data.

PATCH	/api/interactive/[id]/slides/[slideid]	Update element overrides (position, size, content) for a single slide.
POST	/api/interactive/[id]/export/pdf	Trigger asynchronous PDF export via Trigger.dev background job.
POST	/api/interactive/[id]/export/video	Trigger asynchronous video export with Remotion rendering.
GET	/api/interactive/[id]/export/status	Poll export job status and retrieve download URL when complete.
POST	/api/voiceover/generate	Generate an ElevenLabs voiceover for a single slide script.
GET	/api/voiceover/[id]	Poll voiceover generation status; returns R2 download URL on completion.

All mutating routes (POST, PATCH, DELETE) validate the request body against a Zod schema before processing. If validation fails, the route returns a 400 response with a structured error object describing which fields failed. This pattern was introduced early in the internship after code review bots flagged several routes that accepted unvalidated JSON bodies.

Export routes (PDF and video) return immediately with a job ID after triggering the Trigger.dev background task. The client polls the `/api/interactive/[id]/export/status` endpoint until the status field changes to 'completed' or 'failed'. On completion, the status response includes a signed R2 download URL.

The voiceover routes follow the same async pattern. Voiceovers are queued individually per slide. The browser-based presentation player polls the database for each slide's voiceover status and begins playback when the status changes to 'ready'.

5. Final Status and Remaining Follow-Up

By the end of the internship, the main planned workstreams had either been completed or converted into clearly scoped follow-up areas. Several tasks that were originally listed as remaining work, such as analytics and AI assistant improvements, were implemented during the later weeks. Other items, such as Composio integration and quiz certificates, remain appropriate future additions rather than blockers for the completed internship scope.

5.1 What Was Delivered

The table below summarises the main deliverables completed during the internship. It is not a full replacement for the implementation chapter; rather, it provides a compact overview of the product value and technical scope that had been delivered by the end of the internship period.

Deliverable	Description	Status / Value
Projects and Spaces	Project-level workspaces for conversations, source files, generated presentations, custom instructions, fonts, and colour settings.	Implemented; improves organisation and repeatable enterprise workflows.
Interactive Slides	Browser-based animated presentations with structured slide JSON, multiple templates, responsive rendering, voiceovers, and hosted playback.	Implemented; largest product expansion during the internship.
Quiz System	Quiz slide generation, result handling, in-deck feedback, and database support for learning-oriented presentations.	Implemented; adds training and onboarding value.
Interactive Editor	Inline text editing, table editing, chart editing, image gallery, image cropping, undo support, and AI-assisted slide changes.	Implemented; turns generated output into an editable product.

Export Pipeline	PDF and video export flows using background jobs, generated assets, R2 storage, signed URLs, and status polling.	Implemented; supports professional delivery outside the browser.
Sharing and Analytics	Access controls, public/private sharing, invitations, viewer events, quiz analytics, CSV export, and live refresh behaviour.	Implemented; supports enterprise distribution and reporting.
Internationalisation	Dashboard, generation flow, generated content, and email-template localisation for English and Slovak users.	Implemented; supports Agency's domestic and international usage.
Custom AI Presentations	A separate custom-generation path with structured contracts, progress feedback, and a hybrid editor model.	Implemented; supports bespoke enterprise presentation requests.
Reliability, Security, and Quality	Credit-reservation fixes, tier-resolution fixes, signed-URL handling, cache invalidation work, route validation, manual tests, unit tests, and AI-assisted review.	Implemented throughout; improves production safety and maintainability.

5.2 How the Project Can Be Used After the Internship

The work completed during the internship will continue to be used by Agency as a production product. The projects and spaces module gives enterprise users a stable way to organise client-specific or department-specific presentation work. The interactive slides system provides a hosted format that can be distributed as a link, used in training or sales contexts, and improved over time through analytics. The editor, quiz system, voiceovers, exports, and sharing controls together make the product suitable for workflows where the presentation must be generated, refined, delivered, and measured within the same platform.

This is especially relevant because Agency is selling the platform mostly to enterprise clients as a service. Enterprise customers typically need repeatability, access control, branding consistency, reporting, and reliable delivery more than isolated experimentation with a new AI feature. The internship work directly supports those needs: project-level instructions and sources improve consistency, hosted

interactive presentations reduce delivery friction, analytics provide evidence of engagement, and PDF or video export allows the output to fit into existing business processes.

The architecture also leaves room for continued commercial development. New templates, additional analytics dimensions, certificate generation, external tool integrations, and collaborative editing can be added on top of the same core data model. In this sense, the internship endpoint is not only a set of completed features, but a foundation for the next version of the service.

5.3 Remaining Follow-Up Areas

Some areas remain suitable for further development after the internship. These follow-up areas are additive rather than blocking: the implemented platform already supports the main generation, editing, sharing, export, and analytics flows, while the items below would deepen the product for more specialised enterprise scenarios.

Area	Final Status	Notes
Composio Integration	Future follow-up	External tool connectors remain useful, but were not required for the completed internship deliverables.
Share and Analytics	Implemented	Public/private sharing, invitations, view analytics, quiz analytics, CSV export, and live refresh behaviour were added.
Quiz Certificates and Configuration	Future follow-up	Quiz generation and result handling were implemented; certificates and advanced configuration remain future improvements.
AI Assistant Editor Improvements	Substantially implemented	The assistant gained stronger validation, repair handling, tool coverage, theme editing, and better grounding on current slide state.
Custom AI Presentations	Initial product version implemented	Agent-generated bespoke presentations, hybrid editing, publishing validation, and export work were introduced in Weeks 15–16.

The Composio integration remains a future extension. It would use the Composio SDK to connect the AI assistant to external services such as calendars, email, and CRM systems. This work is largely additive and does not require a redesign of the current generation pipeline.

The analytics panel was implemented as part of the share and analytics workstream. It records viewing and quiz-related events, shows recent viewer activity, and

provides quiz result information. Further improvements could add deeper engagement metrics and more advanced segmentation.

Quiz certificates remain a future enhancement. The intended approach is to generate a PDF certificate from a React-based template, personalise it with the viewer's name and score, and provide a download link from the quiz results slide. Additional configuration options could include pass thresholds, attempt limits, and whether correct answers are shown after submission.

The AI assistant editor improvements were substantially completed. I tightened the per-slide-type field guide, added validation and repair for malformed tool calls, expanded the assistant's tool coverage, and grounded responses on the resolved post-edit slide state. Further work could continue improving the assistant's ability to make complex multi-slide edits safely.

6. Future Steps

This chapter outlines directions for continued development beyond the internship period. These are areas that were identified during the internship as valuable but fell outside the agreed scope or could not be fitted into the remaining time.

6.1 Real-Time Collaborative Editing

The current editor is single-user. Only one person can edit a presentation at a time, and concurrent edits from two browser sessions can produce conflicting overrides. A natural next step is to add real-time collaboration using a CRDT-based approach (for example, Yjs) or operational transforms over a WebSocket layer. This would allow teams to co-edit a presentation simultaneously, which is a common workflow in sales and training teams.

The overrides model already stores edits as a flat map of element IDs to positions and content, which is well-suited to merge-friendly conflict resolution. Adding a Yjs document per presentation, synced via a websocket server (for example, Liveblocks or a self-hosted y-websocket), would be a feasible path given the existing data model.

6.2 Advanced Engagement Intelligence

The implemented analytics system captures core viewing and quiz events. A more advanced system could extend this with time-on-slide, device and location data, and more detailed viewer segmentation. This would help presenters understand which slides hold attention, which sections are skipped, and which questions cause difficulty for viewers.

This data could also feed back into the AI generation process. If analytics data from previous presentations on the same topic is available, the model could be instructed to spend more time on high-dropout slides and simplify slides with poor engagement.

6.3 White-Label and Embedding

Customers may ask whether presentations could be embedded in their own websites or delivered under a custom domain. The current architecture already serves presentations from a single domain with a shared presentation player. Supporting custom domains would require subdomain routing at the Vercel edge, a domain verification flow in the user settings, and custom CSS theming applied at the domain level rather than the presentation level.

An embed mode would be simpler: a small JavaScript snippet that loads the presentation player in an iframe with a `postMessage` API for controlling playback. This pattern is well-established and would not require significant infrastructure changes.

6.4 Multi-Modal Input

The current generation flow accepts text prompts and uploaded documents. A natural extension is to accept other input types: voice input (transcribed to text before generation), images (analysed for content), and screen recordings (transcribed from audio and combined with slide screenshots). This would allow users to generate a presentation from a recorded demo or meeting without any manual writing.

OpenAI's audio and vision APIs are already available and would integrate straightforwardly with the existing generation pipeline. The main work would be in the user interface: recording controls, upload previews for images, and a transcription status indicator for longer recordings.

7. Evaluation

This chapter evaluates the internship work from a practical and technical perspective. The project was not a controlled research experiment, but a production-oriented software engineering assignment in which priorities, user expectations, and implementation details changed repeatedly as the platform evolved. For that reason, the evaluation focuses on delivery, adaptation, reliability, and the lessons learned from building real product functionality under changing conditions.

7.1 What Went Well

One of the strongest outcomes of the internship was my ability to adapt to frequent changes in product direction. Several features changed shape after discussion with the internship mentor, after testing, or after new commercial priorities appeared. Instead of treating these changes as interruptions, I adjusted the implementation plan and used the existing architecture to extend the platform in a controlled way. This was visible in the evolution from static exports toward interactive slides, then toward quizzes, editor improvements, sharing, analytics, custom presentation flows, and production reliability work.

The amount of code and product functionality delivered during the internship was also significant. I contributed across the front end, API layer, database schema, storage layer, AI generation pipeline, export system, internationalisation, and testing workflow. This broad contribution added high practical value because the features were not isolated prototypes; they connected into a product flow that Aigency can present and sell to clients. The interactive slides ecosystem in particular changed the platform from a static presentation generator into a more complete service for creating, editing, publishing, and measuring browser-based presentations.

The use of AI-assisted code review and development support also worked well when combined with manual judgement. Automated feedback helped identify missing validation, authorisation gaps, race conditions, and maintainability issues. However, the useful part was not simply receiving suggestions, but checking them against the actual codebase and deciding which changes improved the product without adding unnecessary complexity.

7.2 What Was Difficult

The most difficult part of the internship was managing the dependency between many interconnected features. A change in the editor could affect export behaviour; a change in access control could affect sharing; a change in generation schema could affect rendering, analytics, or AI-assisted edits. This meant that many tasks required more than implementing the visible user interface. They also required checking persistence, validation, billing, cache invalidation, background jobs, and asset access.

Production reliability issues were also challenging because they appeared in edge cases rather than in the basic happy path. Examples included credit reservations

that had to be released correctly after failed jobs, tier-resolution mistakes that could expose paid functionality, signed-URL problems for private R2 assets, cache invalidation after editor changes, and rate limits during voiceover generation. These issues made the project more demanding, but they also made the internship more valuable because they reflected the kind of problems that occur in real software products.

Another challenge was validating AI-generated output. The platform depends on structured data produced by language models, but the model can still return incomplete, malformed, or semantically inconsistent fields. This required defensive schemas, repair logic, stricter tool contracts, and careful client-side handling so that AI output could be transformed into reliable application state instead of being trusted blindly.

7.3 Lessons Learned

The main lesson from the internship is that AI product development still requires conventional software engineering discipline. Prompt quality and model choice matter, but the product becomes reliable only when the surrounding system validates inputs, handles failures, isolates long-running work, protects private assets, records useful state, and gives users a predictable editing experience. In practice, the most important engineering work was often at the boundaries between AI generation and normal application infrastructure.

I also learned the importance of incremental delivery. Large features such as interactive slides, video export, and the editor became manageable when they were split into smaller contracts: schema design, rendering, persistence, editor overrides, background processing, and user feedback. This approach made it easier to adapt when requirements changed, because each layer could be improved without replacing the entire system.

Finally, the internship strengthened my understanding of technical communication. Many implementation decisions involved tradeoffs between speed, reliability, user experience, and maintainability. Explaining those tradeoffs clearly to the internship mentor, documenting them in pull requests, and reflecting them in this realization document was part of the engineering work itself, not a separate administrative task.

7.4 Design Tradeoffs and Rejected Alternatives

Several alternatives were considered or implicitly rejected during the internship. One alternative was to keep the product focused mainly on static exports such as PPTX and PDF. That would have been simpler, but it would not match the product direction described by LectureGuru: turning ideas and documents into professional learning material, narrated slides, videos, and shareable outputs. The browser-based interactive slide system was therefore the more valuable direction, even though it created additional work in rendering, editing, sharing, analytics, and export compatibility.

A second rejected approach was to treat generation as one large prompt that directly produces the final presentation. This was easier conceptually, but fragile for longer presentations and difficult to validate. The structured planning and slide-generation approach made the output easier to check, persist, edit, repair, and export. The same principle applied to the editor assistant: model output could not simply be trusted as free-form data, because it becomes application state.

A third important tradeoff concerned asset access. Raw public URLs would have been easier to implement, but they would not match the expectations of an enterprise-oriented service. The safer design was to keep assets in R2 and expose them through controlled access paths: signed download URLs where appropriate and same-origin proxy routes for media such as interactive-slide audio or images when playback requirements make that preferable.

Finally, video export required a deliberate rendering choice. A simple browser capture workflow would have been easier to prototype, but it would be harder to make consistent for animated React-based slides and generated audio. The Remotion and Trigger.dev-based rendering approach added infrastructure, but it fit the requirement that exported MP4 output should preserve the browser presentation as a professional deliverable.

7.5 Quality and Risk Management

Quality in this project meant more than whether a feature worked once during a demonstration. A generated presentation had to remain editable, exportable, shareable, billable, and measurable under realistic conditions. Risk management therefore became part of the implementation work: AI output had to be validated, background jobs had to report their status, private assets had to stay protected, analytics had to be scoped, and edited slides had to remain consistent across the browser, database, and export pipeline.

Risk	Mitigation already present	Recommended next step
Malformed or incomplete AI-generated slide/editor data	Structured generation, Zod validation, repair logic, stricter editor tool contracts, and autosave validation reduce the chance of invalid application state.	Maintain a small regression set of representative prompts and editor operations that can be rerun after schema, prompt, or tool changes.
Failed export, audio, image, or generation jobs after credits are reserved	Credit usage and reservation tables, release paths, cleanup tasks, and Trigger.dev status handling reduce the risk of silently consuming credits after failed work.	Add or expand integration tests for failed PDF, video, audio, image, and generation jobs so billing recovery remains reliable.

Private generated or uploaded assets becoming inaccessible or overexposed	R2-backed storage is accessed through signed download URLs or same-origin proxy routes depending on file type, with authenticated access paths for protected presentations.	Review asset access routes periodically as sharing, embedding, custom domains, and long-lived media playback grow.
Analytics and sharing collecting or exposing more data than intended	Share slugs, visibility controls, invitation flows, viewer tracking, quiz attempt storage, and analytics exports are scoped to presentations and organisation access patterns.	Define a privacy-aware analytics policy before adding deeper segmentation, device, location, or engagement intelligence.
Editor output drifting from generated content or exported files	Editor overrides, cache revalidation, post-edit state grounding, and export-specific rendering paths keep browser, database, and export output aligned.	Introduce regression checks that compare edited browser slides with exported PDF/video output for representative layouts.

7.6 Competencies Demonstrated

The internship demonstrates several expected competencies. I analysed product and business problems, translated them into technical requirements, designed database and API changes, implemented full-stack features, integrated AI services, handled background jobs, protected private assets, improved billing reliability, and documented the resulting system in a way that can be reviewed.

It also demonstrates the ability to work in a changing product environment. Many tasks required balancing speed with maintainability, and user-facing polish with production safety. The work therefore required technical implementation, communication, testing, code reviews, and reflection on tradeoffs. These are the areas where the internship developed my skills most strongly.

8. Conclusion

Over sixteen weeks at Aigency, I completed a substantial and varied body of work. The most significant deliverable was the interactive slides ecosystem, including the browser-based player, quiz system, in-browser editor, sharing and analytics features, and PDF and video export pipeline. Together, these features changed the platform from a static presentation generator into a more complete interactive presentation product. In addition, the projects and spaces feature improved organisation for power users, the select-choices workflow improved generation quality, the Custom AI work introduced a new bespoke presentation path, and the internationalisation rollout made the product more accessible to Slovak-speaking customers.

The technical scope was broader than expected at the start. Billing infrastructure, security hardening, background job management, cache invalidation, asset access, and concurrency issues all became important while building features that initially appeared mostly front-end oriented. Resolving issues such as the credit-reservation leak, tier-resolution bug, R2 signed-URL problems, stale cache behaviour, and voiceover rate limits gave me practical experience with production systems rather than isolated feature development.

Working with AI code review bots as part of the pull-request workflow was consistently valuable. The bots identified authorisation gaps, missing input validation, potential race conditions, and maintainability problems that could have been missed during manual review. Running an automated review pass before every pull request became a regular quality step rather than an optional addition.

The objectives formulated in the project plan were met and, in several areas, exceeded. This was especially clear in the interactive slides module, quiz system, share and analytics functionality, editor improvements, and Custom AI presentation work. Some follow-up areas remain, particularly around deeper collaboration, advanced analytics, certificates, and external integrations, but the core internship objectives were completed and extended into production-ready product functionality. The future directions in Chapter 6 provide a clear set of next priorities for continued development.

Reference List

- Next.js Documentation. Vercel, Inc. <https://nextjs.org/docs>
- Drizzle ORM Documentation. <https://orm.drizzle.team/docs>
- Supabase Documentation. Supabase, Inc. <https://supabase.com/docs>
- Vercel AI SDK Documentation. Vercel, Inc. <https://sdk.vercel.ai/docs>
- OpenAI API Reference. OpenAI. <https://platform.openai.com/docs>
- ElevenLabs API Documentation. ElevenLabs. <https://docs.elevenlabs.io>
- Remotion Documentation. <https://www.remotion.dev/docs>
- Cloudflare R2 Documentation. Cloudflare, Inc. <https://developers.cloudflare.com/r2>
- Unsplash API Documentation. <https://unsplash.com/documentation>
- next-intl Documentation. <https://next-intl.dev/docs>
- Trigger.dev Documentation. <https://trigger.dev/docs>
- Composio SDK Documentation. <https://docs.composio.dev>
- Yjs Collaborative Editing Framework. <https://docs.yjs.dev>
- Playwright Documentation. Microsoft. <https://playwright.dev/docs>
- pdf-lib Documentation. <https://pdf-lib.js.org>

Generative AI Acknowledgment

Generative AI tools were used during the internship as development and writing support tools. Claude Opus 4.7 and GPT-5.5 were used for code assistance, debugging support, architectural discussion, code review preparation, and drafting or refining parts of this realization document. Their use was especially helpful for scaffolding implementation ideas, comparing possible technical approaches, improving the clarity of technical explanations, and checking whether report sections were understandable to an academic reader.

The tools were not used as a replacement for project ownership. AI-generated code was read, edited, tested, and integrated manually before being accepted.

Suggestions were checked against the actual Aigency codebase, the implemented database schema, the production behaviour of the platform, and the feedback received during the internship. When a generated suggestion did not match the real architecture or created unnecessary complexity, it was discarded or rewritten.

Representative uses included asking for possible approaches to validating structured AI output, drafting utility functions for schema repair, reviewing API route security concerns, improving wording in technical report sections, and checking whether a paragraph explained a production issue clearly enough for a reader who had not worked on the codebase. In each case, the final decision and final wording remained my responsibility.

The main limitation of these tools is that they do not have independent knowledge of the live product, the company priorities, or the actual behaviour of the system after deployment. They can produce plausible technical explanations that still need to be verified. For that reason, all AI-assisted work was validated through manual testing, code review, comparison with repository evidence, and alignment with the internship objectives. The responsibility for the contents of this document and for the implemented work remains mine.